

JUNIT ATIVIDADE: TESTES PARAMETRIZADOS (1)

Questão 1

Estenda a classe de testes criada no exemplo de autenticação do usuário para que os diferentes valores para autenticação sejam fornecidos usando `@ValueSource`;

```
class UsuarioAuthParametrizadoTest {
    private Usuario usuario; 6 usages

    @BeforeEach
    void setUp() {
        usuario = new Usuario();
        usuario.setNome("Usuario Teste");
        usuario.setEmail("teste@dominio");
        usuario.setSenha("senhaSegura123");
    }

    @ParameterizedTest
    @ValueSource(strings = {"senhaSegura123"})
    void testAutenticacaoSucessoParametrizada(String senhaValida) {
        assertDoesNotThrow(() -> usuario.autenticar(senhaValida));
    }

    @ParameterizedTest
    @ValueSource(strings = {"senhaErrada", "123", " ", "wrong_pass"})
    void testAutenticacaoFalhaParametrizada(String senhaInvalida) {
        assertThrows(Exception.class, () -> usuario.autenticar(senhaInvalida));
    }
}
```

Questão 2

Crie uma sobrecarga do método de autenticação que permita informar o algoritmo a ser usado (ex: "SHA-256", "MD5", "SHA-1"), testando o lançamento de exceções caso o algoritmo seja desconhecido (o que também deve ser testado);

```

public void autenticar(String senhaEntrada, String algoritmo) throws ExceededAttemptsException {
    long agora = System.currentTimeMillis();

    if (agora < bloqueadoAte) {
        throw new ExceededAttemptsException();
    }

    validarString(senhaEntrada);

    String hashEntrada = gerarHash(senhaEntrada, algoritmo);

    if (hashEntrada.equals(this.senhaHash)) {
        falhasTimestamp.clear();
        bloqueadoAte = 0;
    } else {
        registrarFalha(agora);
    }
}
}

```

```

@Test
void testAutenticacaoAlgoritmoInvalido() {
    assertThrows(IllegalArgumentException.class, () -> {
        usuario.autenticar(senhaEntrada: "senhaSegura123", algoritmo: "ALGORITMO-INEXISTENTE");
    });
}

@ParameterizedTest
@ValueSource(strings = {"SHA-256", "MD5", "SHA-1"})
void testSobrecargaAutenticacaoAlgoritmos(String algoritmo) {
    if (algoritmo.equals("SHA-256")) {
        assertDoesNotThrow(() -> usuario.autenticar(senhaEntrada: "senhaSegura123", algoritmo));
    } else {
        assertThrows(Exception.class, () -> usuario.autenticar(senhaEntrada: "senhaSegura123", algoritmo));
    }
}
}

```

Questão 3

Crie os casos de teste para a autenticação anterior, informando os algoritmos com duas versões alternativas: @ValueSource e @EnumSource;

```

@ParameterizedTest
@ValueSource(strings = {"SHA-256", "MD5", "SHA-1"})
@DisplayName("Teste de algoritmos via @ValueSource")
void testAutenticacaoAlgoritmosValueSource(String algoritmo) {
    if (algoritmo.equals("SHA-256")) {
        assertDoesNotThrow(() -> usuario.autenticar(senhaEntrada: "senhaSegura123", algoritmo));
    } else {
        assertThrows(Exception.class, () -> usuario.autenticar(senhaEntrada: "senhaSegura123", algoritmo));
    }
}

@ParameterizedTest
@EnumSource(AlgoritmoHash.class)
@DisplayName("Teste de algoritmos via @EnumSource")
void testAutenticacaoAlgoritmosEnumSource(AlgoritmoHash algoritmo) {
    String nomeAlgoritmo = algoritmo.getNome();
    if (nomeAlgoritmo.equals("SHA-256")) {
        assertDoesNotThrow(() -> usuario.autenticar(senhaEntrada: "senhaSegura123", nomeAlgoritmo));
    } else {
        assertThrows(Exception.class, () -> usuario.autenticar(senhaEntrada: "senhaSegura123", nomeAlgoritmo));
    }
}

```

Questão 4

Crie testes alternativos para que os valores informados sejam carregados usando @CsvSource e, depois, @CsvFileSource.

```

@ParameterizedTest
@CsvSource({
    "senhaSegura123, SHA-256, true",
    "senhaIncorreta, SHA-256, false",
    "senhaSegura123, MD5, false",
    "senhaSegura123, SHA-1, false"
})
@DisplayName("Teste de autenticação via @CsvSource")
void testAutenticacaoCsvSource(String senha, String algoritmo, boolean sucessoEsperado) {
    if (sucessoEsperado) {
        assertDoesNotThrow(() -> usuario.autenticar(senha, algoritmo));
    } else {
        assertThrows(Exception.class, () -> usuario.autenticar(senha, algoritmo));
    }
}

@ParameterizedTest
@CsvFileSource(resources = "/dados_autenticacao.csv", numLinesToSkip = 1)
@DisplayName("Teste de autenticação via @CsvFileSource")
void testAutenticacaoCsvFileSource(String senha, String algoritmo, boolean sucessoEsperado) {
    if (sucessoEsperado) {
        assertDoesNotThrow(() -> usuario.autenticar(senha, algoritmo));
    } else {
        assertThrows(Exception.class, () -> usuario.autenticar(senha, algoritmo));
    }
}

```

Questão 5

Compare as várias versões de testes criadas e analise as vantagens e desvantagens de cada uma.

1. @ValueSource

- **Vantagem:** Extrema simplicidade para testes com apenas um argumento.
- **Desvantagem:** Limitado a um único parâmetro por execução.

2. @EnumSource

- **Vantagem:** Segurança de tipos (Type-safety) e centralização das opções válidas, evitando erros de digitação.
- **Desvantagem:** Exige a criação e manutenção de um Enum específico.

3. @CsvSource

- **Vantagem:** Permite testar múltiplos argumentos relacionados (ex: entrada, algoritmo e resultado esperado) de forma legível dentro do código.
- **Desvantagem:** Torna o código poluído se houver dezenas de cenários de teste.

4. @CsvFileSource

- **Vantagem:** Melhor opção para grandes volumes de dados (Big Data) e permite que analistas de negócio editem os testes sem mexer no código Java.
- **Desvantagem:** Requer o gerenciamento de arquivos externos e caminhos de diretório.